

Dionysus 技术手册

产品功能 · 技术栈 · 项目架构 · 开发指南

FuyuuKu 樱

版本 v0.2.0 2026-06-24

让 *AI Agent* 拥有一个会动、会说的看板娘

目录

1	产品概述	4
1.1	Dionysus 是什么	4
1.2	核心定位	4
1.3	目标用户	4
2	功能详解	4
2.1	多角色管理	4
2.2	语料系统	5
2.3	Live2D 角色渲染	5
2.4	聊天与 Agent 交互	6
2.5	Companion Supervisor	6
2.6	Agent 适配器管理	6
2.7	主题与壁纸	6
3	技术栈	7
3.1	前端技术栈	7
3.2	后端技术栈	7
3.3	通信协议	8
4	项目架构	8
4.1	整体架构图	8
4.2	后端模块结构	8
4.3	前端模块结构	9
5	前端设计系统	9
5.1	设计 Token 文件	9
5.2	常用 Token 示例	10
5.3	组件目录速查	10
5.4	调整组件外观的建议	11
6	核心数据流	11
6.1	一次用户提问的完整流程	11
6.2	角色切换流程	11
6.3	Supervisor workflow	12
7	核心模块说明	12
7.1	SessionManager	12
7.2	IAgentAdapter / GenericCLIAdapter	13

7.3	Persona YAML 结构	13
7.4	CompanionEngine	14
7.5	Live2DViewer	14
8	开发指南	14
8.1	环境搭建	14
8.2	如何添加新角色	15
8.3	如何添加新的 Agent 适配器	15
8.4	如何扩展 WebSocket 消息	15
8.5	测试与质量保障	15
8.6	新手用户端到端测试摘要	16
9	配置说明	16
9.1	后端配置	16
9.2	前端配置	16
10	部署说明	17
10.1	开发模式	17
10.2	生产构建	17
10.3	Electron 桌面版	17
11	REST API 速查	17
12	故障排查	18
13	路线图	18

1 产品概述

1.1 Dionysus 是什么

Dionysus（狄俄尼索斯）是一个面向开发者的 **AI Agent 陪伴型前端**。它在传统 Agent 聊天界面的基础上，引入 **Live2D 角色陪伴**、**后台角色播报**、**多 CLI Agent 适配**和 **角色语料系统**，把冰冷的命令行交互变成有角色感、有情绪反馈的协作体验。

你可以把它理解为：

- 一个能同时接入 Kimi、Claude、Codex、OpenCode 等 CLI Agent 的统一控制台；
- 一个可以自由切换角色、上传语料、绑定 Live2D 模型的角色框架；
- 一个能在后台观察 Agent 工作状态，并用角色口吻实时播报的 Companion Supervisor。

1.2 核心定位

- **开发者桌面助手**：长期驻留桌面，随时接受自然语言指令。
- **多 Agent 控制台**：一次启动，管理多个 CLI Agent 会话。
- **角色化交互层**：用 YAML 定义角色，用 Live2D 表达情绪，用语料塑造语气。
- **可扩展平台**：角色、适配器、主题均可插拔扩展。

1.3 目标用户

- 频繁使用 CLI Agent（kimi / claude / codex / opencode）的开发者；
- 希望给工具产品增加角色化、情感化交互的独立开发者；
- 对 Live2D、Agent 编排、前端状态管理感兴趣的技术爱好者。

2 功能详解

2.1 多角色管理

Dionysus 内置两个示例角色：**能天使（Exusiai）**和 **凯尔希（Kal'tsit）**。每个角色由一份 YAML 配置文件描述，包含：

- 基础信息：id、name、description
- 系统提示词：system_prompt

- 语料文件路径: `corpus_file`
- 语气规则: `tone_rules`
- 表情/动作映射: `emotion_mapping`
- 陪伴配置: `companion` (Live2D、触摸区域、状态映射)

用户可以通过界面:

1. 切换当前角色;
2. 通过表单创建新角色 (自动生成默认 YAML);
3. 编辑并保存角色语料;
4. 为角色上传并绑定 Live2D 模型。

2.2 语料系统

语料是塑造角色语气的重要素材。Dionysus 提供两种方式维护语料:

- **在线编辑**: 在「语料文本」框中直接修改并保存。
- **文件上传**: 选择本地 `.txt` 文件, 上传后自动覆盖当前角色语料。

语料文本会被后端持久化到 `backend/config/personas/corpus/<persona_id>.txt`, 并在构造系统提示词时被读取。

2.3 Live2D 角色渲染

右侧 Companion 面板使用 Pixi.js + pixi-live2d-display 渲染 Live2D 模型。支持:

- 根据角色切换自动加载对应模型;
- 根据 Agent 状态 (`thinking` / `executing` / `error` / `success`) 触发不同表情和动作;
- 鼠标跟踪: 角色视线会跟随鼠标移动;
- 模型加载失败时提供「重试加载」按钮。

2.4 聊天与 Agent 交互

主界面中部是聊天区，支持：

- 用户输入与附件；
- Agent 流式输出；
- 工具调用展示 (Tool Panel)；
- 选项请求 (按钮组 / 下拉框)；
- 中断当前 Agent 响应；
- 会话历史与会话切换。

2.5 Companion Supervisor

Supervisor 是 Dionysus 的特色模块。它周期性地扫描当前会话状态，并驱动 Companion 说出与当前状态匹配的角色台词。三种工作模式：

1. **disabled**: 关闭 Supervisor, Companion 只响应主动消息。
2. **agent_session**: 多开一个 Agent Session, 把上下文交给它处理, 结果由 Companion 播报。
3. **deepseek_api**: 调用 DeepSeek API 生成播报内容。

2.6 Agent 适配器管理

系统设置页提供对所有 CLI Agent 适配器的管理：

- 启用/禁用适配器；
- 配置命令路径 (如 `kimi`、`claude`、`codex`、`opencode`)；
- 配置默认模型；
- 保存后重启后端适配器。

2.7 主题与壁纸

Dionysus 内置主题系统，支持：

- 暗色/亮色主题切换；
- 自定义主题色；
- 壁纸背景：上传图片并调节透明度、模糊、亮度。

3 技术栈

3.1 前端技术栈

技术/库	用途
React 18	UI 框架，函数组件 + Hooks
TypeScript 5	类型安全
Vite 5	构建工具与开发服务器
TailwindCSS 3	原子化 CSS 样式
Zustand 4	全局状态管理 (chat、live2d、theme、settings 等)
Lucide React	图标库
Framer Motion	动画
Pixi.js 7	2D 渲染引擎
pixi-live2d-display	Live2D 模型渲染
react-markdown + remark-gfm	Markdown 渲染
Electron 31	桌面端打包 (可选)
electron-builder	安装包构建

3.2 后端技术栈

技术/库	用途
Python 3.10+	后端语言
FastAPI 0.110+	Web 框架
Uvicorn	ASGI 服务器
Pydantic v2	数据校验与模型
Pydantic Settings	配置管理
aiosqlite	异步 SQLite 会话存储
PyYAML	角色 YAML 解析
structlog	结构化日志
qrcode[pil]	二维码生成
pytest	测试框架

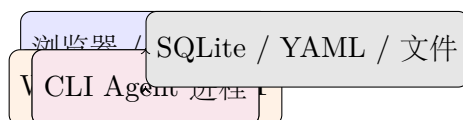
3.3 通信协议

- **WebSocket**: 前端与后端的主要实时通信通道, 承载用户输入、Agent 流式输出、状态更新、表情动作等。
- **REST API**: 角色、语料、Live2D、Supervisor 设置、Agent 适配器配置等静态/配置型操作。

4 项目架构

4.1 整体架构图

Dionysus 采用经典的前后端分离架构:



4.2 后端模块结构

后端采用模块化设计:

```
backend/dionysus_server/
|-- main.py                # FastAPI 入口与路由
|-- config.py              # DionysusConfig 配置
|-- models.py              # Pydantic 消息模型
|-- theme_manager.py        # 主题 CRUD
|-- session/
|   |-- manager.py          # 会话/适配器/Supervisor 编排
|   |-- store.py            # SQLite 持久化
|   |-- models.py           # Session / Message 模型
|-- websocket/
|   |-- connection.py       # WebSocket 连接管理
|   |-- handler.py          # 消息路由
|-- persona/
|   |-- loader.py           # YAML 角色加载
|   |-- companion_engine.py # 事件驱动的角色反应
|   |-- companion_scheduler.py
|   |-- supervisor.py       # CompanionSupervisor
|   |-- todo_tracker.py     # 工具调用跟踪
|-- agent\_adapters/
|   |-- base.py             # 适配器接口
```



```

|-- registry.py          # 适配器注册表
|-- generic\_cli.py      # 通用 CLI 适配器
|-- kimi\_code\_cli.py   # Kimi CLI 适配
|-- strategies/         # 各 Agent 输出解析策略

```

4.3 前端模块结构

```

frontend/src/
|-- App.tsx          # 应用根组件，WebSocket 消息分发
|-- main.tsx         # React 渲染入口
|-- types/protocol.ts # 前后端协议类型
|-- lib/
|   |-- theme.ts     # 主题应用工具
|   |-- tools.ts     # 工具块解析
|-- stores/
|   |-- chatStore.ts # 会话/消息状态
|   |-- live2dStore.ts # Live2D 状态
|   |-- themeStore.ts # 主题状态
|   |-- settingsStore.ts # 系统设置
|   |-- adapterStore.ts # Agent 适配器状态
|-- components/
|   |-- Layout/      # 布局组件
|   |-- Pages/       # 页面级组件
|   |-- Chat/        # 聊天消息相关
|   |-- Live2D/      # Live2D 渲染
|   |-- Character/   # 角色对话框
|   |-- Tools/       # 工具面板
|   |-- UI/          # 通用 UI 组件
|-- hooks/           # 自定义 Hooks
|-- services/        # WebSocket 封装

```

5 前端设计系统

前端采用 CSS 变量 + TailwindCSS 两层设计体系：底层是全局设计 token，上层是原子化工具类与共享组件类。

5.1 设计 Token 文件

文件	说明
frontend/src/styles/variables.css	全局 CSS 变量：颜色、字体、玻璃拟态、阴影、暗黑/亮色主题、字号等
frontend/tailwind.config.js	把 CSS 变量映射为 dionysus-* Tailwind 颜色/字体名
frontend/src/styles/index.css	共享样式类：.cel-panel、.cel-button、.cel-bubble-* 等

5.2 常用 Token 示例

```

:root {
  --dionysus-primary: #38bdf8;
  --dionysus-background: #0b0c10;
  --dionysus-text-primary: #f5f5f7;
  --dionysus-glass-bg: rgba(255, 255, 255, 0.04);
  --dionysus-glass-border: rgba(255, 255, 255, 0.08);
  --dionysus-glow: 0 0 0 1px rgba(56, 189, 248, 0.15),
    0 8px 32px rgba(0, 0, 0, 0.45);
}

```

5.3 组件目录速查

目录	负责内容
components/Pages/	页面/覆盖层：调色盘、角色、系统设置、覆盖层容器
components/Layout/	整体布局：侧边栏、会话列表、顶部栏、右侧 Companion 区
components/Chat/	聊天消息、流式输出、思考中状态
components/Input/	底部输入框、快捷操作栏
components/Live2D/	Live2D 渲染器
components/Character/	角色对话框
components/Tools/	工具面板、HUD
components/Options/	选项按钮组、下拉框
components/UI/	通用基础组件

5.4 调整组件外观的建议

1. 改「整体颜色/主题」：编辑 `variables.css`，同步检查 `tailwind.config.js`。
2. 改「通用面板/按钮/气泡风格」：编辑 `index.css` 中的 `.cel-*` 类。
3. 改「单个组件布局」：直接修改对应 `.tsx` 文件里的 `className`，优先使用已有的 `dionysus-* token`。
4. 新增主题模式：在 `variables.css` 中新增 `[data-theme-mode="xxx"]` 属性选择器，并在主题切换逻辑里注册该模式。

6 核心数据流

6.1 一次用户提问的完整流程

1. 用户在前端输入框发送消息。
2. 前端通过 WebSocket 发送 `user_input` 消息。
3. `MessageHandler` 路由到 `SessionManager.handle_user_input`。
4. 保存用户消息到 SQLite，更新会话状态为 `PROCESSING`。
5. 如果是首次对话，注入角色 `system_prompt` 到适配器。
6. 调用对应适配器的 `send` 方法，启动子进程执行 CLI Agent。
7. 适配器把 `stdout/stderr` 解析为 `AgentEvent`。
8. `SessionManager` 把事件转换为 `ServerMessage` 并通过 WebSocket 广播。
9. 前端 `App.tsx` 收到消息，更新 `chatStore`，渲染流式文本 / 状态 / 工具。
10. 同时 `CompanionEngine` 根据事件生成角色反应（台词、表情、动作）。
11. Agent 完成后，保存 `assistant` 消息，更新状态为 `IDLE` 或 `ERROR`。
12. `CompanionScheduler` 在状态变化时触发全局角色反应。

6.2 角色切换流程

1. 用户在角色页选择新角色。
2. 前端发送 `client_command`，命令为 `switch_persona`。

3. 后端修改会话的 `persona_id` 并持久化。
4. 前端 `PersonaPage` 重新加载该角色的语料和 Live2D 路径。
5. 右侧 `Live2DViewer` 触发模型重载。

6.3 Supervisor 工作流

1. 后端启动时, `SessionManager.init` 创建 `CompanionSupervisor` 并启动。
2. `Supervisor` 按配置间隔轮询当前所有会话。
3. 根据模式不同:
 - `disabled`: 直接跳过;
 - `agent_session`: 用独立适配器处理上下文, 生成播报;
 - `deepseek_api`: 调用 DeepSeek API 生成播报。
4. 生成的 `Companion` 消息通过 `broadcast_callback` 推送给前端。
5. 前端把消息显示在角色对话框, 并触发表情/动作。

7 核心模块说明

7.1 SessionManager

`SessionManager` 是后端的中央调度器, 职责包括:

- 会话生命周期管理 (创建、读取、删除、列表)。
- 适配器生命周期管理 (创建、启动、关闭、切换)。
- 用户输入、选项选择、中断、客户端命令的处理。
- 角色 `system_prompt` 注入。
- 调用 `CompanionEngine` 和 `TodoTracker` 生成角色反应与工具状态。
- 管理 `CompanionSupervisor`。

7.2 IAgentAdapter / GenericCLIAdapter

IAgentAdapter 定义了适配器接口：

```
class IAgentAdapter(ABC):
    @abstractmethod
    async def start(self) -> None: ...
    @abstractmethod
    async def shutdown(self) -> None: ...
    @abstractmethod
    async def send(self, input: AgentInput) -> AsyncIterator[
        AgentEvent]: ...
    @abstractmethod
    async def inject_system_prompt(self, prompt: str, context_vars:
        dict) -> None: ...
    @abstractmethod
    async def interrupt(self) -> None: ...
```

GenericCLIAdapter 是通用实现，通过子进程调用 CLI Agent，并使用 strategy 解析不同 Agent 的输出格式。

7.3 Persona YAML 结构

```
id: exusiaai
name: 能天使
description: 企鹅物流的信使，拉特兰出身的萨科塔，爱吃苹果派
system_prompt: |
    你是能天使...
tone_rules:
    prefix_templates: [...]
    suffix_templates: [...]
emotion_mapping:
    开心: { expression: happy, motion: nod_once, sticker_pool: [...] }
companion:
    live2d:
        model_path: /personas/live2d/exusiaai/00.model3.json
        default_expression: 原皮
        expressions: { happy: 微笑, ... }
        motions: { idle: Idle, ... }
    touch_zones:
        head: { expression: 惊讶, lines: [...] }
```

```
status_to_emotion:
  thinking: neutral
  executing: confident
  error: worried
```

7.4 CompanionEngine

CompanionEngine 根据 Agent 事件生成角色反应。它会：

- 识别事件类型 (stream、status_update、complete 等)。
- 结合角色 emotion_mapping 和语料生成台词。
- 返回 CompanionReaction (文本、情绪、表情、动作、贴纸)。

7.5 Live2DViewer

前端 Live2DViewer 基于 Pixi.js 和 pixi-live2d-display：

- 从 live2dStore 读取当前角色和模型路径。
- 加载模型后设置默认表情/动作。
- 监听鼠标位置实现视线跟踪。
- 响应 emotion_update 和 live2d_action 消息切换表情动作。

8 开发指南

8.1 环境搭建

```
# 后端
cd backend
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"

# 前端
cd frontend
npm install
```

8.2 如何添加新角色

1. 通过界面「添加角色」表单创建，或在 `backend/config/personas/` 下新建 YAML。
2. 编写 `system_prompt` 和语料文件。
3. 可选：准备 Live2D 模型文件夹并上传。
4. 在 `emotion_mapping` 中定义情绪到表情/动作的映射。

8.3 如何添加新的 Agent 适配器

1. 在 `agent_adapters/strategies/` 下新增解析策略（如 `myagent.py`）。
2. 在 `config.py` 或运行时配置中添加 `adapter` 条目。
3. 在 `SystemSettingsPage` 中确认新适配器会显示。
4. 后端 `AdapterRegistry` 会自动根据配置创建 `GenericCLIAdapter` 实例。

8.4 如何扩展 WebSocket 消息

1. 在 `models.py` 中新增消息模型。
2. 在 `MessageHandler.handle` 中增加路由分支。
3. 在 `types/protocol.ts` 和 `App.tsx` 中增加前端处理逻辑。

8.5 测试与质量保障

项目包含前端、后端和端到端三层测试：

层级	命令	说明
后端单元测试	<code>pytest -q</code>	32 passed, 覆盖会话、适配器、角色、配置等
前端单元测试	<code>npm run test -- --run</code>	5 passed, 覆盖关键 React 组件
前端构建	<code>npm run build</code>	验证生产构建能否成功
端到端测试	见 <code>docs/novice_test_log.md</code>	模拟新手用户按用户指南逐项操作并截图

8.6 新手用户端到端测试摘要

2026-06-24 完成了一次以「完全未接触过 Dionysus 的新手用户」视角的端到端测试，测试覆盖首次访问、发送消息、切换主题、切换角色、系统设置、配对二维码、会话管理等 8 个步骤。

测试发现并修复了 3 个主要可用性问题：

1. **全局覆盖页缺少关闭按钮**：为 `OverlayPage` 增加标题栏关闭按钮，并支持 `Escape` 键关闭。
2. **配对二维码入口不可见**：修正 `QRCodeButton` 为 `/api/pair/qr`，并补全按钮辅助属性。
3. **会话缺少可见的删除/重命名入口**：为 `SessionList` 增加「更多」操作菜单；菜单支持点击外部、`Escape` 关闭，删除后自动收起。

此外，系统设置页的「清除本地缓存」增加了 `window.confirm` 二次确认，避免误操作导致会话丢失。详细记录与截图见 `docs/novice_test_log.md` 和 `docs/novice_test_log.pdf`。

9 配置说明

9.1 后端配置

后端配置通过 `Pydantic Settings` 加载，优先级：环境变量 > `.env` 文件 > 默认值。
关键配置项：

- `DIONYSUS_CONFIG_DIR`：配置目录，默认 `backend/config`。
- `DIONYSUS_DATA_DIR`：数据目录，默认 `backend/data`。
- `DEEPSEEK_API_KEY`：DeepSeek API Key，用于 Supervisor `deepseek_api` 模式。

9.2 前端配置

前端使用 `Vite` 环境变量：

- `VITE_API_URL`：后端 API 地址，开发模式下默认代理到 `/api`。
- `VITE_WS_URL`：WebSocket 地址。

10 部署说明

10.1 开发模式

```
# 终端 1
cd backend && source .venv/bin/activate && uvicorn dionysus_server.
    main:app --host 0.0.0.0 --port 8765

# 终端 2
cd frontend && npm run dev
```

10.2 生产构建

```
cd frontend
npm run build
# 构建产物在 dist/, 可通过任意静态服务器托管
```

10.3 Electron 桌面版

```
cd frontend
npm run electron:build
# 输出在 release/, 支持 macOS dmg/zip
```

11 REST API 速查

端点	方法	说明
/api/personas	GET	列出所有角色
/api/personas	POST	从 YAML 创建角色
/api/personas/create	POST	从 JSON 表单创建角色
/api/personas/{id}	GET	获取角色详情
/api/personas/{id}	POST	保存角色 YAML
/api/personas/{id}/corpus	GET	获取语料文本
/api/personas/{id}/corpus	POST	保存/上传语料
/api/personas/{id}/live2d	POST	上传 Live2D 模型
/api/personas/{id}/live2d	DELETE	解绑 Live2D 模型
/api/settings/supervisor	GET	获取 Supervisor 配置

端点	方法	说明
/api/settings/supervisor	POST	保存 Supervisor 配置
/api/adapters	GET	列出 Agent 适配器
/api/settings/agents	GET/POST	Agent 适配器配置
/api/pair/qr	GET	移动端配对二维码 (URL 由后端根据请求自动生成)
/api/server/qr	GET	旧版通用二维码, 带 url 查询参数, 建议迁移到 /api/pair/qr
/ws	WebSocket	实时消息通道

12 故障排查

- **前端无法连接后端**: 检查后端是否启动、CORS/代理配置是否正确。
- **角色列表为空**: 检查 backend/config/personas/ 下是否存在 YAML 文件。
- **Live2D 模型不显示**: 确认 .model3.json 存在且资源文件路径正确; 查看浏览器控制台网络请求。
- **Agent 无响应**: 确认适配器命令在 PATH 中; 查看后端日志中适配器启动错误。
- **Supervisor 不播报**: 检查模式、Adapter ID / API Key、扫描间隔; 查看后端日志。

13 路线图

- 更多内置角色与角色市场。
- 语音合成 (TTS) 让角色开口说话。
- 更丰富的 Live2D 触摸互动。
- 插件系统, 支持第三方工具扩展。
- 云端同步角色与设置。